Aman Verma

Jaunpur, UP, India | +91 6386617608 | akverma4aman@gmail.com | linkedin.com/in/amanverma765 | github.com/amanverma-765

November 2, 2026

Parker Cox & Engineering Hiring Team
AI & Platform Engineering
Pavago LLC
Meridian, ID, USA (Remote)

Dear Parker and the Pavago Engineering Team,

Pavago's talent marketplace shifts global recruitment from multi-week agency lag to sub-minute vetted matches across 300,000 candidates, establishing a high bar for automated candidate screening and profile retrieval. Delivering reliable candidate evaluation at this scale requires low-latency inference pipelines, deterministic skill-to-evidence parsing, and strict defenses against ungrounded keyword stuffing. Having engineered production multi-turn agent pipelines and empirical retrieval evaluation benchmarks, my experience directly addresses the operational challenges of building fast, cost-efficient AI screening systems that maintain high precision.

In my open-source benchmark **ncert-rag-evals**, I constructed an empirical evaluation testbed across 17 STEM textbooks in **Python** and **Jupyter Notebooks**, implementing custom hybrid search in **SQLite FTS5** and **ChromaDB** to profile 8 retrieval strategies on raw CPU hardware. By fusing BM25 lexical search with dense embeddings via Reciprocal Rank Fusion (RRF), I proved an empirical **86.9% Recall@5 in 8.9ms on CPU**, matching LLM query expansion while cutting latency by **99.8%**. Furthermore, in **vectorless-rag**, I benchmarked dense vector search against LLM-based page indexing using **Pydantic AI** and **Ollama**, mapping the **2.7x token cost** and **2x latency** trade-offs while streaming document windows via **PyMuPDF** with bounded memory (**under 200MB**).

Production screening architectures require rigorous statistical validation and zero tolerance for prompt injection. To ensure defensibility in **ncert-rag-evals**, I validated retrieval significance using paired McNemar statistical hypothesis tests (**p = 0.0021**) supported by 92 unit tests in **Pytest**. Additionally, during my software engineering internship at **LexoLegal**, I built 'Munshi', an autonomous assistant in **Pydantic AI** and **FastAPI**, isolating verified user identity inside AgentDeps rather than LLM tool parameters to eliminate prompt injection risks and orchestrating **Celery** workers with atomic **Redis** distributed locks to handle concurrent messages with **sub-2s latency**.

Bringing this evaluation rigor, agentic architecture background, and pipeline optimization to Pavago will directly accelerate your AI screening and candidate matching workflows. Operating fully aligned with U.S. business hours from India, I am available immediately for full-time remote engineering and look forward to discussing how these systems can drive measurable improvements in your matching infrastructure.

Sincerely,

Aman Verma